T3000 API & Protocol

Graphics · built Nov 13, 2026

Graphics

Visual interface for monitoring and controlling building systems.

Overview

Graphics provide intuitive visual representations of equipment, floor plans, and system diagrams with real-time data overlay.

Graphic Features

- **Floor Plans**: Building layouts with equipment locations
- **System Diagrams**: HVAC system schematics
- **Equipment Graphics**: Individual equipment views
- **Real-Time Data**: Live values displayed on graphics

Graphic Elements

- **Shapes**: Rectangles, circles, lines
- **Text**: Labels and values
- **Images**: Equipment photos, logos
- **Controls**: Buttons, sliders
- **Animations**: Moving/blinking elements based on status

Creating Graphics

1. Navigate to **Graphics** page
2. Click **New Graphic**
3. Add shapes and elements
4. Link data points to graphics
5. Save and publish

Best Practices

- Keep graphics simple and clear

- Use consistent colors and symbols
- Label all elements clearly
- Test on different screen sizes

Next Steps

- Dashboard - Main dashboard
- Monitoring - Data monitoring

Graphics

Graphics Rendering

SVG Graphics Engine

TYPESCRIPT

```typescript
interface GraphicElement {
  id: string;
  type: 'rect' | 'circle' | 'line' | 'text' | 'image';
  x: number;
  y: number;
  width?: number;
  height?: number;
  fill?: string;
  stroke?: string;
  dataBinding?: string;  // Point ID to bind
}

class GraphicsRenderer {
  private svg: SVGSVGElement;
  private elements = new Map<string, GraphicElement>();

  render(graphic: Graphic) {
    this.svg = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
    this.svg.setAttribute('viewBox', `0 0 ${graphic.width} ${graphic.height}`);

    for (const element of graphic.elements) {
      const svgEl = this.createElement(element);
      this.svg.appendChild(svgEl);

      if (element.dataBinding) {
        this.bindData(svgEl, element.dataBinding);
      }
    }

    return this.svg;
  }

  updateValue(pointId: string, value: number) {
    // Find elements bound to this point
```

```
    const elements = Array.from(this.elements.values()).filter(
      e => e.dataBinding === pointId
    );

    for (const element of elements) {
      this.updateElement(element, value);
    }
  }
}
```

Real-Time Data Binding

TYPESCRIPT

```
class GraphicsDataBinder {
  private ws: WebSocket;
  private renderer: GraphicsRenderer;

  constructor(renderer: GraphicsRenderer) {
    this.renderer = renderer;
    this.ws = new WebSocket('ws://localhost:9103/ws');

    this.ws.onmessage = (event) => {
      const data = JSON.parse(event.data);

      if (data.type === 'point-update') {
        this.renderer.updateValue(data.point, data.value);
      }
    };
  }

  subscribeToPoints(pointIds: string[]) {
    this.ws.send(JSON.stringify({
      type: 'subscribe',
      points: pointIds
    }));
  }
}
```

Next Steps

- REST API

- WebSocket API